

面向对象第二天：

回顾：

1. 什么是类？什么是对象？
2. 如何创建类？如何创建对象？如何访问成员？
3. this：指代当前对象，哪个对象调用方法它指的就是哪个对象
 - this.成员变量名-----访问成员变量

当成员变量与局部变量同名时，若想访问成员变量，则this不能省略
4. 构造方法：
 - 给成员变量赋初值、与类同名、没有返回值类型(连void都没有)，创建(new)对象时被自动调用。若自己
不写构造方法，则默认一个无参构造，若自己写了构造方法，则不再默认提供。

精华笔记：

1. 继承：
 - 作用：代码复用
 - 通过extends实现继承
 - 超类/基类/父类：共有的属性和行为
 - 派生类/子类：特有的属性和行为
 - 派生类可以访问：超类的+派生类的，超类不能访问派生类的
 - 一个超类可以有多个派生类，一个派生类只能有一个超类-----单一继承
 - 具有传递性
 - java规定：构造派生类之前必须先构造超类
 - 在派生类的构造方法中若没有调用超类的构造方法，则默认super()调用超类的无参构造方法
 - 在派生类的构造方法中若自己调用了超类的构造方法，则不再默认提供

注意：super()调用超类构造方法，必须位于派生类构造方法中的第1行
2. super：指代当前对象的超类对象
 - super.成员变量名-----访问超类的成员变量
 - 当超类成员变量和派生类成员变量同名时，super指超类的，this指派生类的

若没有同名现象，则写super和this是一样的
 - super.方法名()-----调用超类的方法
 - super()-----调用超类的构造方法
3. 方法的重写(override/overriding)：重新写、覆盖

- 发生在父子类中，方法名相同，参数列表相同
- 重写方法被调用时，看对象的类型-----new谁就调谁的，这是规定

```
class 餐馆{
    void 做餐(){ 做中餐 }
}
//1)我还是想做中餐-----不需要重写
class Aoo extends 餐馆{
}
//2)我想改做西餐-----需要重写
class Boo extends 餐馆{
    void 做餐(){ 做西餐 }
}
//3)我想在中餐基础之上加入西餐-----需要重写(先super中餐，再加入西餐)
class Coo extends 餐馆{
    void 做餐(){
        super.做餐();
        做西餐
    }
}
```

4. final：最终的、不能改变的-----单独应用几率低

- 修饰变量：变量不能被改变
- 修饰方法：方法不能被重写
- 修饰类：类不能被继承

笔记：

1. 继承：

- 作用：代码复用
- 通过extends实现继承
- 超类/基类/父类：共有的属性和行为
派生类/子类：特有的属性和行为
- 派生类可以访问：超类的+派生类的，超类不能访问派生类的
- 一个超类可以有多个派生类，一个派生类只能有一个超类-----单一继承
- 具有传递性

```
public class Person {
    String name;
    int age;
    String address;
    void eat(){
        System.out.println(name+"正在吃饭...");
    }
    void sleep(){
        System.out.println(name+"正在睡觉...");
    }
}
```

```

    }
    void sayHi(){
        System.out.println("大家好, 我叫"+name+", 今年"+age+"岁了, 家住"+address);
    }
}

public class Student extends Person{
    String className;
    String stuId;
    Student(){
    }
    Student(String name,int age,String address,String className,String stuId){
        super.name = name;
        super.age = age;
        super.address = address;
        this.className = className;
        this.stuId = stuId;
    }

    void study(){
        System.out.println(name+"正在学习...");
    }
}

public class Teacher extends Person {
    double salary;
    Teacher(){
    }
    Teacher(String name,int age,String address,double salary){
        this.name = name;
        this.age = age;
        this.address = address;
        this.salary = salary;
    }

    void teach(){
        System.out.println(name+"正在讲课...");
    }
}

public class Doctor extends Person {
    String title;
    Doctor(){
    }
    Doctor(String name,int age,String address,String title){
        this.name = name;
        this.age = age;
        this.address = address;
        this.title = title;
    }

    void cut(){
        System.out.println(name+"正在做手术...");
    }
}

public class ExtendsTest {

```

```

public static void main(String[] args) {
    Student zg = new Student();
    zg.name = "张光";
    zg.age = 23;
    zg.address = "廊坊";
    zg.className = "jsd2311";
    zg.stuId = "001";
    zg.eat();
    zg.sleep();
    zg.sayHi();
    zg.study();

    Student ll = new Student("李林", 22, "佳木斯", "jsd2311", "002");
    ll.eat();
    ll.sleep();
    ll.sayHi();
    ll.study();

    Teacher zl = new Teacher("赵亮", 36, "山东", 6000.0);
    zl.eat();
    zl.sleep();
    zl.sayHi();
    zl.teach();

    Doctor wpf = new Doctor("王鹏飞", 45, "山西", "主任医师");
    wpf.eat();
    wpf.sleep();
    wpf.sayHi();
    wpf.cut();

    //演示超类不能访问派生类的
    Person p = new Person();
    p.name = "人";
    p.age = 1;
    p.address = "未知";
    p.eat();
    p.sleep();
    p.sayHi();
    //p.stuId = "005"; //编译错误, 超类不能访问派生类的
}
}

```

o java规定：构造派生类之前必须先构造超类

- 在派生类的构造方法中若没有调用超类的构造方法，则默认super()调用超类的无参构造方法
- 在派生类的构造方法中若自己调用了超类的构造方法，则不再默认提供

注意：super()调用超类构造方法，必须位于派生类构造方法中的第1行

```

public class Person {
    String name;
    int age;
    String address;
}

```

```

    Person(){
    }
    Person(String name,int age,String address){
        this.name = name;
        this.age = age;
        this.address = address;
    }

    void eat(){
        System.out.println(name+"正在吃饭...");
    }
    void sleep(){
        System.out.println(name+"正在睡觉...");
    }
    void sayHi(){
        System.out.println("大家好, 我叫"+name+", 今年"+age+"岁了, 家住"+address);
    }
}

public class Student extends Person{
    String className;
    String stuId;
    Student(){
    }
    Student(String name,int age,String address,String className,String stuId){
        super(name,age,address);
        this.className = className;
        this.stuId = stuId;
    }

    void study(){
        System.out.println(name+"正在学习...");
    }
}

public class Teacher extends Person{
    double salary;
    Teacher(){
    }
    Teacher(String name,int age,String address,double salary){
        super(name,age,address);
        this.salary = salary;
    }

    void teach(){
        System.out.println(name+"正在讲课...");
    }
}

public class Doctor extends Person {
    String title;

```

```

    Doctor(){
    }
    Doctor(String name,int age,String address,String title){
        super(name,age,address);
        this.title = title;
    }
    void cut(){
        System.out.println(name+"正在做手术...");
    }
}

```

2. super: 指代当前对象的超类对象

- super.成员变量名-----访问超类的成员变量

当超类成员变量和派生类成员变量同名时，super指超类的，this指派生类的
若没有同名现象，则写super和this是一样的

- super.方法名()-----调用超类的方法
- super()-----调用超类的构造方法

```

public class SuperDemo {
    public static void main(String[] args) {
        Boo o = new Boo();
    }
}

//1)在派生类的构造方法中若没有调用超类的构造方法，则默认super()调用超类的无参构造方法
class Aoo{
    Aoo(){
        System.out.println("超类构造方法");
    }
}
class Boo extends Aoo{
    Boo(){
        super(); //默认的，调用超类的无参构造方法
        System.out.println("派生类构造方法");
    }
}

//2)在派生类的构造方法中若自己调用了超类的构造方法，则不再默认提供
class Coo{
    Coo(int a){
    }
}
class Doo extends Coo{
    Doo(){
        super(5); //调用超类的有参构造
    }
    /*
    //如下代码为默认的：
    Doo(){
        super();
    }
    */
}

```

```

    }
    */
}

```

3. 方法的重写(override/overriding): 重新写、覆盖

- 发生在父子类中, 方法名相同, 参数列表相同
- 重写方法被调用时, 看对象的类型-----new谁就调谁的, 这是规定

```

class 餐馆{
    void 做餐(){ 做中餐 }
}
//1)我继承餐馆后还是想做中餐-----不需要重写
class Aoo extends 餐馆{
}
//2)我继承餐馆后想改做西餐-----需要重写
class Boo extends 餐馆{
    void 做餐(){ 做西餐 }
}
//3)我继承餐馆后想在中餐基础之上加入西餐-----需要重写(先super中餐, 再加入西餐)
class Coo extends 餐馆{
    void 做餐(){
        super.做餐();
        做西餐
    }
}

```

```

public class Person {
    String name;
    int age;
    String address;
    Person(){
    }
    Person(String name,int age,String address){
        this.name = name;
        this.age = age;
        this.address = address;
    }

    void eat(){
        System.out.println(name+"正在吃饭...");
    }
    void sleep(){
        System.out.println(name+"正在睡觉...");
    }
    void sayHi(){
        System.out.println("大家好, 我叫"+name+", 今年"+age+"岁了, 家住"+address);
    }
}

public class Student extends Person{
    String className;
}

```

```

String stuId;
Student(){
}
Student(String name,int age,String address,String className,String stuId){
    super(name,age,address); //传递的是name/age/address的值
    this.className = className;
    this.stuId = stuId;
}

void study(){
    System.out.println(name+"正在学习...");
}

void sayHi(){
    System.out.println("大家好, 我叫"+name+", 今年"+age+"岁了, 家住"+address+", 所在班级为:"+className+", 学号为: "+stuId);
}
}

public class Teacher extends Person{
    double salary;
    Teacher(){
    }
    Teacher(String name,int age,String address,double salary){
        super(name,age,address);
        this.salary = salary;
    }

    void teach(){
        System.out.println(name+"正在讲课...");
    }

    void sayHi(){
        System.out.println("大家好, 我叫"+name+", 今年"+age+"岁了, 家住"+address+", 工资为:"+salary);
    }
}

public class Doctor extends Person {
    String title;
    Doctor(){
    }
    Doctor(String name,int age,String address,String title){
        super(name,age,address);
        this.title = title;
    }
    void cut(){
        System.out.println(name+"正在做手术...");
    }
}

public class OverrideDemo {
    public static void main(String[] args) {

```



```

//重写方法被调用时，看对象的类型执行方法(new谁就调谁)
Student zg = new Student("张光",23,"廊坊","jsd2309","001");
zg.sayHi(); //调用Student重写之后的
Teacher ll = new Teacher("李林",35,"佳木斯",6000.0);
ll.sayHi(); //调用Teacher重写之后的
Doctor zl = new Doctor("赵亮",45,"山东","主任医师");
zl.sayHi(); //调用Person中的
    }
}

```

4. final：最终的、不能改变的-----单独应用几率低

- 修饰变量：变量不能被改变

```

class Eoo{
    final int a = 5;
    int b = 6;
    void test(){
        //a = 55; //编译错误，final修饰的变量不能被改变
        b = 66;
        final int c = 8;
        //c = 88; //编译错误，final修饰的变量不能被改变
    }
}

```

- 修饰方法：方法不能被重写

```

class Foo{
    final void show(){}
    void test(){}
}
class Goo extends Foo{
    //void show(){} //编译错误，final修饰的方法不能被重写
    void test(){}
}

```

- 修饰类：类不能被继承

```

final class Hoo{}
//class Ioo extends Hoo{} //编译错误，final修饰的类不能被继承
class Joo{}
final class Koo extends Joo{} //正确，不能当老爸，但能当儿子

```

5. 综合练习：

```

public class Animal {
    String name;
    int age;
    String color;
    Animal(){
    }
}

```

```

Animal(String name,int age,String color){
    this.name = name;
    this.age = age;
    this.color = color;
}
void drink(){
    System.out.println(color+"色的"+age+"岁的"+name+"正在喝水...");
}
void eat(){ //明天继续做处理
    System.out.println(color+"色的"+age+"岁的"+name+"正在吃饭...");
}
}

public class Dog extends Animal {
    Dog(){
    }
    Dog(String name,int age,String color){
        super(name,age,color);
    }
    void lookHome(){
        System.out.println(color+"色的"+age+"岁的狗狗"+name+"正在看家...");
    }
    void eat(){
        System.out.println(color+"色的"+age+"岁的狗狗"+name+"正在吃肯头...");
    }
}

public class Chick extends Animal {
    Chick(){
    }
    Chick(String name,int age,String color){
        super(name,age,color);
    }
    void layEggs(){
        System.out.println(color+"色的"+age+"岁的小鸡"+name+"正在下蛋...");
    }
    void eat(){
        System.out.println(color+"色的"+age+"岁的小鸡"+name+"正在吃小米...");
    }
}

public class Fish extends Animal {
    Fish(){
    }
    Fish(String name,int age,String color){
        super(name,age,color);
    }
    void eat(){
        System.out.println(color+"色的"+age+"岁的小鱼"+name+"正在吃小虾...");
    }
}

public class AnimalTest {

```

```

public static void main(String[] args) {
    Dog dog = new Dog("小黑",2,"黑");
    dog.eat();
    dog.drink();
    dog.lookHome();

    Chick chick = new Chick("小花",1,"花");
    chick.eat();
    chick.drink();
    chick.layEggs();

    Fish fish = new Fish("小金",2,"金");
    fish.eat();
    fish.drink();
}
}

```

补充:

1. 泛化：从程序设计角度而言叫泛化，从代码实现角度而言叫继承，泛化就是继承
2. 继承要符合is(是)的关系
3. 继承的是父类的成员变量和普通方法，不包括构造方法，父类的构造方法是被子类通过super来调用的

```

class Aoo{
    int a;
    Aoo(){
    }
    void show(){
    }
}
class Boo extends Aoo{
    继承了Aoo类的a+show(), 并没有继承Aoo类的构造方法
}

```

4. 重写与重载的区别：-----常见面试题
 - 重写(overriding)：发生在父子类中，方法名相同，参数列表相同
 - 重载(overloading)：发生在同一类中，方法名相同，参数列表不同
5. 重载特例情况：

```

class Aoo{
    void show(){
    }
}
class Boo extends Aoo{ //在此类中发生了show()的重载
    void show(String name){
    }
}

```

6. 明日单词：

- 1) swim: 游泳
- 2) abstract: 抽象的
- 3) interface: 接口
- 4) implements: 实现